

Configuring your project with Autoconf

The main purpose of using Autoconf is to generate a *configure* script suitable for the target system. Why we need a *configure* script is for portability. Although all Linux variants are similar, they aren't identical. They have slightly different sets of header files, different lists of functions in the system libraries, etc. So, it's the developer's responsibility to check and resolve all these differences via a configuration script. The *configure* script generally performs different system-level capability checks, including machine information (host-name, version, etc), programs (*bison*, *lex*, etc), libraries, header files, *typedefs*, structures, compiler characteristics and library functions. Let's see how we can create a *configure* script using *autoconf*, with the simplest possible code:

```
/* hello_world.c */
#include<stdio.h>
int main(void)
{
    printf("hello world\n");
    return 0;
}
```

Our *Makefile* will look something like this:

```
all: hello_world
clean:
    rm -rf hello_world
```

Now run the following commands:

```
$ autoscan
$ ls
autoscan.log configure.scan hello_world.c Makefile
$ mv configure.scan hello_world.c
$ autoheader
$ ls
autom4te.cache autoscan.log config.h.in configure.ac hello_world.c
Makefile
$ autoconf
$ ls
autom4te.cache autoscan.log config.h.in configure configure.ac
hello_world.c Makefile
```

As you can see, *autoconf* has completed its work; *configure* and *config.h.in* have been created, as shown in Figure 1. (Files and commands in brackets in the figure are optional.) Autoconf includes the programs *autoheader*, *autom4te*, *autoreconf*, *autoupdate* and *ifnames*. You can check the *autoconf* manual to learn about each program.

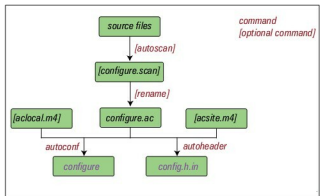


Figure 1: A data flow diagram for Autoconf

Now you are ready to run *./configure*:

```
$ ./configure
checking for gcc... gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... no
checking for suffix of object files... o
checking whether we are using the GNU C compiler... yes
checking whether gcc accepts -g... yes
checking for gcc option to accept ISO C89... none needed
configure: creating ./config.status
config.status: error: cannot find input file: 'Makefile.in'
$ ls
autom4te.cache autoscan.log config.h.in config.log config.status
configure configure.ac hello_world.c Makefile
```

Wait! You can't run *configure* yet because it needs *Makefile.in* as input. How do we get *Makefile.in*? Let's go a step further.

Using Automake

We can use *automake* to generate *Makefile.in* automatically, which in turn creates a portable *Makefile* (refer to Figure 2). First, create a *Makefile.am*; this can't be done automatically, so use the simplest *Makefile.am* of two lines:

```
bin_PROGRAMS = hello_world
hello_world_SOURCES = hello_world.c
```

Edit *configure.ac* and add *AM_INIT_AUTOMAKE* below the *AC_INIT* directive. Next, use the following commands:

```
$ aclocal
$ automake --add-missing
$ touch NEWS README AUTHORS ChangeLog
$ ls
aclocal.m4 autom4te.cache ChangeLog config.log configure COPYING
hello.c install-sh missing README AUTHORS autoscan.log config.h.in
```